A SYNOPSIS ON

INTRUSION DETECTION SYSTEM (IDS)

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted by:

Pratibha Manral **2104793**

Under the Guidance of

Ms. Shikha Thakur
Assistant Professor

Department of Computer Applications
Graphic Era (Deemed to be University)
Dehradun, Uttarakhand
March - 2026

Graphic Era Deemed to be University
DEHRADUN

CANDIDATE'S DECLARATION

I/we hereby certify that the work which is being presented in the Synopsis entitled **"INTRUSION DETECTION SYSTEM (IDS)"** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Computer Applications in the Department of Computer Applications of the Graphic Era (Deemed to be University), Dehradun shall be carried out by the undersigned under the supervision **Ms. Shikha Thakur, Assistant Professor**, Department of Computer Applications, Graphic Era (Deemed to be University), Dehradun.

Pratibha Manral 2104793

The above mentioned students shall be working under the supervision of the undersigned on the **INTRUSION DETECTION SYSTEM (IDS).**

Signature
Ms. Shikha Thakur

Signature
Head of the Department

Internal Evaluation (By DPRC Committee)

Status of the Synopsis: Accepted / Rejected

Any Comments:

Name of the Committee Members: **Signature with Date**

1.

2.

Table of Contents

Chapter No.	Description	Page No.
Chapter 1	Introduction and Problem Statement	**4**
Chapter 2	Background/ Literature Survey	**6**
Chapter 3	Objectives	**10**
Chapter 4	Hardware and Software Requirements	**12**
Chapter 5	Possible Approach/ Algorithms	**14**
	References	**20**

Chapter 1
Introduction and Problem Statement

In the following sections, a brief introduction and the problem statement for the work has been included.

1.1 Introduction

With the rapid growth of computer networks and internet-connected systems, network security has become an important requirement. Organizations, educational institutions, and individuals continuously exchange data over networks, making them potential targets for unauthorized access, malicious activities, and network-based attacks.

An **Intrusion Detection System (IDS)** is a security mechanism that monitors network traffic and system activities to identify suspicious or potentially malicious behavior. Unlike a firewall, which primarily controls whether traffic is allowed or blocked, an IDS focuses on **detecting and reporting suspicious activities**.

The proposed project focuses on developing a lightweight **Network Intrusion Detection System** using Python. The system captures network packets, extracts relevant information, analyzes traffic patterns, and generates alerts when predefined suspicious behavior is detected.

The project uses **Scapy** for packet analysis, **SQLite** for storing network events and alerts, and a **Tkinter/Web-based dashboard** for displaying security information.

1.2 Problem Statement

While drones have become increasingly popular for photography, delivery, and recreation, their control systems remain largely dependent on physical controllers or mobile apps. These traditional interfaces have limitations:

- **Learning Curve:** Require practice to master joystick controls
- **Physical Dependency**: Need to carry a controller device
- **Accessibility Issues**: Difficult for people with motor impairments
- **Lack of Immersion**: Break the natural connection between user intent and drone movement

This project addresses the problem of creating an **intuitive, contact-free drone control system** using hand gestures, with real-time visual feedback in a simulated 3D environment.

1.3 System Overview

The proposed **Intrusion Detection System** consists of five major components:

1. **Packet Capture Module:**
   Uses Scapy to capture and inspect network packets.
2. **Traffic Analysis Module:**
   Extracts information such as source IP, destination IP, protocol, ports, packet size, and timestamps.
3. **Intrusion Detection Module:**
   Applies predefined rules to identify suspicious traffic patterns such as port scans and abnormal packet rates.
4. **Alert and Database Module:**
   Generates security alerts and stores them in an SQLite database for future analysis.
5. **Monitoring Dashboard:**
   Displays captured traffic, detected alerts, source IP addresses, risk levels, and other security information.

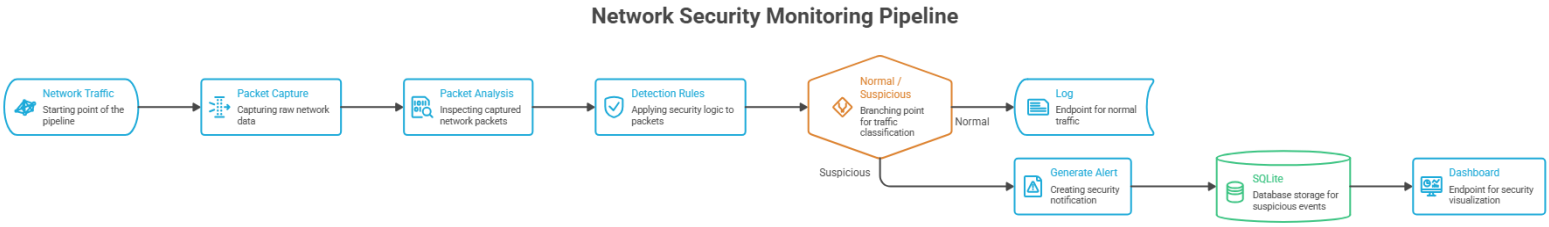


Fig. 1.1 High Level Dataflow

1.4 Need and Significance

- **Real-Time Monitoring:** Helps observe network activity continuously.
- **Early Detection:** Identifies suspicious patterns before they can cause further problems.
- **Automated Alerts:** Reduces the need for continuous manual monitoring.
- **Educational Value:** Demonstrates practical concepts of networking and cybersecurity.
- **Low Cost:** Can be implemented using Python and open-source libraries.
- **Centralized Logging:** Stores detected security events in a database.
- **Extensibility:** Additional detection rules can be added in the future.
- **Visualization:** Provides a dashboard for easier interpretation of network activity.

Chapter 2

Background/ Literature Survey

2.1 Foundations of Intrusion Detection

Intrusion detection has developed alongside computer networks and cybersecurity.

Early intrusion detection approaches primarily relied on manually defined rules and signatures. As network traffic increased, automated monitoring and analysis techniques became necessary.

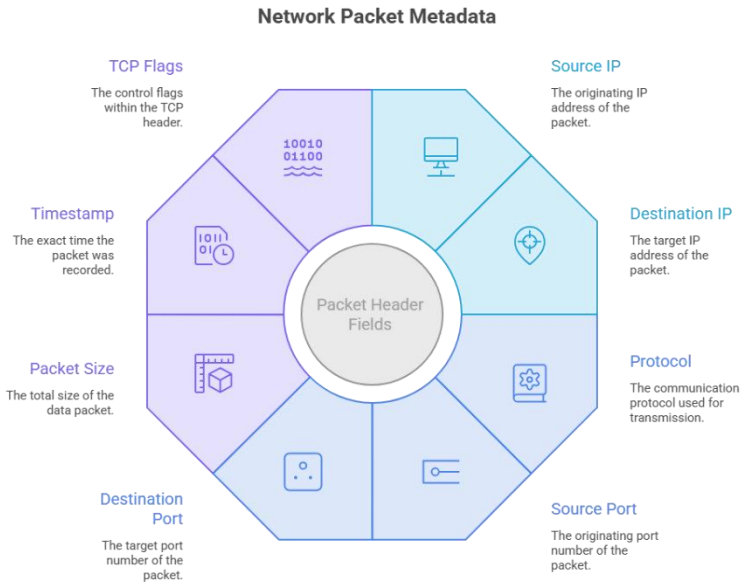
Traditional approaches include:

- **Signature-Based Detection:** Searches for known attack patterns.
- **Anomaly-Based Detection:** Identifies deviations from normal behavior.
- **Rule-Based Detection:** Uses predefined conditions to identify suspicious events.
- **Network-Based IDS:** Monitors traffic moving across a network.
- **Host-Based IDS:** Monitors activities occurring on individual systems.

The proposed project primarily focuses on a network-based, rule-based IDS

2.2 Network Intrusion Detection

A Network Intrusion Detection System monitors network packets and analyzes their characteristics. Typical packet information includes:



By analyzing these values over a period of time, the system can identify unusual patterns.

For example, if one source IP attempts connections to a large number of different destination ports within a short period, the behavior may indicate a possible port-scanning pattern.

2.3 Packet Analysis Using Scapy

Scapy is a Python-based packet manipulation and network analysis library.

It provides functionality for:

- Capturing packets
- Inspecting packet layers
- Reading IP addresses
- Identifying protocols
- Extracting port information
- Analyzing packet characteristics

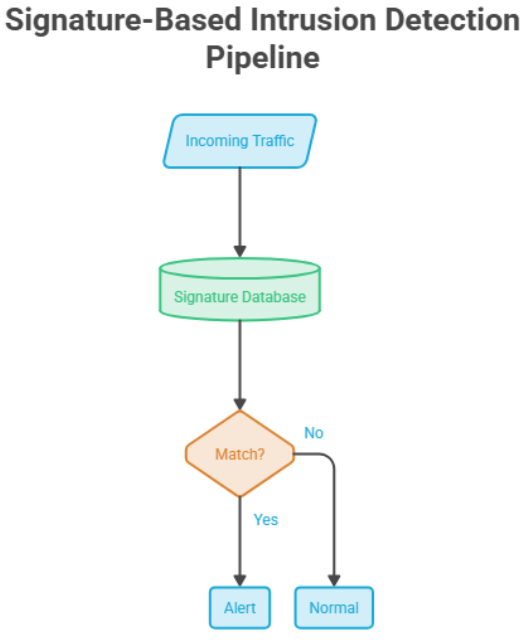
For this project, Scapy acts as the interface between the network traffic and the intrusion detection logic.

2.4 Existing Detection Approaches

Common approaches used in IDS include:

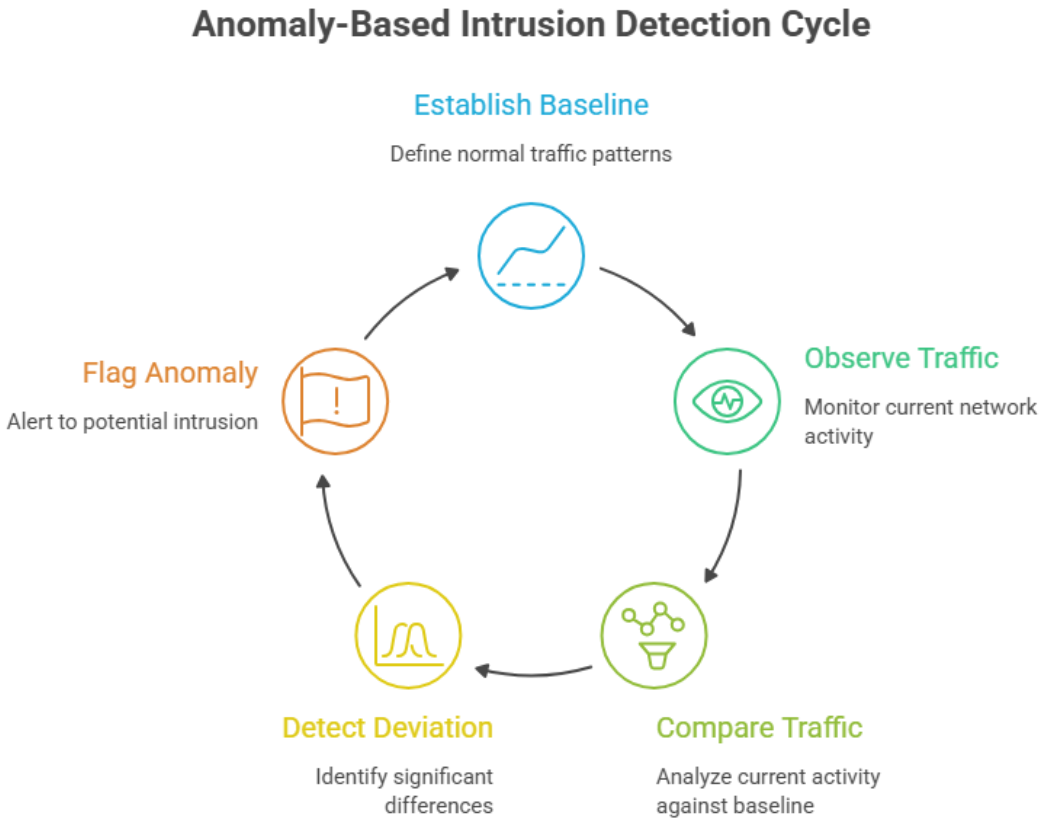
Signature-Based Detection

Known attack signatures are compared against observed traffic.



Anomaly-Based Detection

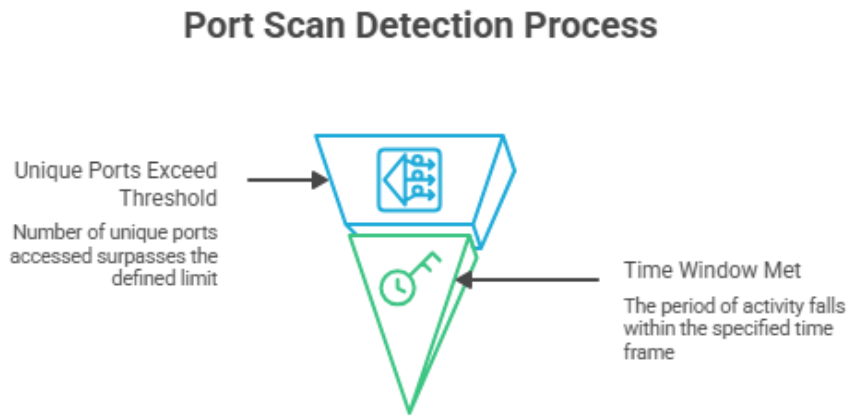
The system establishes expected behavior and detects significant deviations.



Rule-Based Detection

Specific conditions are defined by the administrator.

Example:



The proposed mini-project primarily uses this rule-based approach because it is easier to implement, understand, test, and demonstrate in an academic environment.

2.5 Identified Research Gap

Many basic network monitoring programs only display packet information without automatically interpreting the traffic.

Some systems may also require complex infrastructure or large datasets.

The proposed project focuses on creating a lightweight and understandable IDS prototype that combines:

- Real-time packet capture
- Traffic analysis
- Rule-based intrusion detection
- Risk classification
- Database logging
- Security alerts
- Graphical/web visualization

This makes the system suitable for learning and demonstrating fundamental IDS concepts.

Chapter 3

Objectives

3.1 Primary Objectives

1. **Real-Time Packet Monitoring:** Capture and monitor network packets in real time using Python and Scapy.
2. **Traffic Analysis:** Extract important network information including source IP, destination IP, protocol, ports, packet size, and timestamps.
3. **Port Scan Detection:** Identify suspicious behavior involving connections to multiple destination ports within a short time period.
4. **Abnormal Traffic Detection:** Detect unusually high packet rates from a particular source.
5. **Repeated Connection Detection:** Identify repeated connection attempts between network hosts.
6. **ICMP Traffic Analysis:** Monitor unusual ICMP packet patterns.
7. **Alert Generation:** Generate alerts containing information about the detected event and its risk level.
8. **Database Logging:** Store network events and security alerts using SQLite.

3.2 Secondary Objectives

1. **Risk Classification:** Categorize detected events into LOW, MEDIUM, and HIGH risk levels.
2. **Dashboard:** Provide a graphical interface for viewing network activity and security alerts.
3. **Alert History:** Allow previously detected events to be reviewed.
4. **IP Statistics:** Display information about frequently observed or suspicious IP addresses.
5. **Traffic Statistics:** Display total packets, suspicious packets, and detected alerts.
6. **Export Function:** Provide an option to export security logs for further analysis.

3.3 Scope and Limitations

Scope

- Network packet monitoring
- Detection of predefined suspicious patterns
- Real-time alert generation

- SQLite-based event storage
- Dashboard-based visualization
- Controlled cybersecurity demonstration environment

Limitations

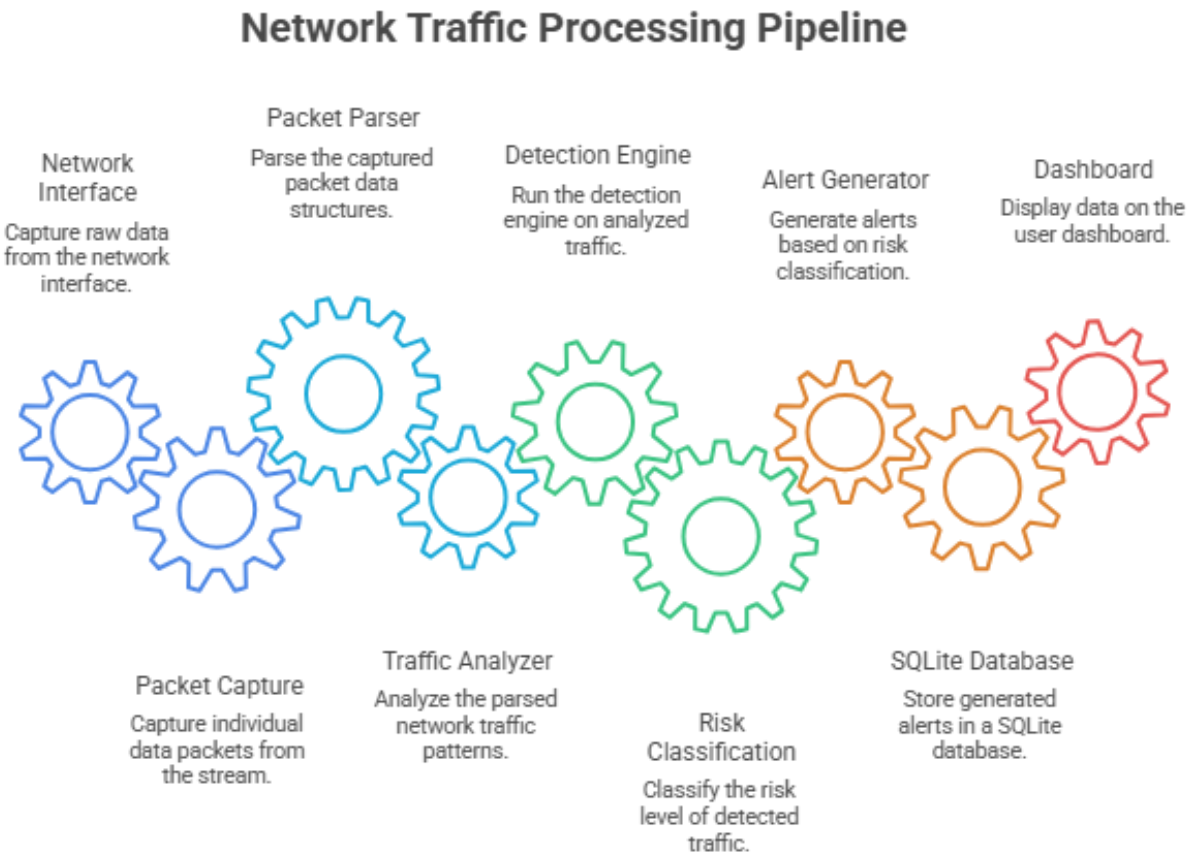
- Detection is limited to predefined rules.
- It may generate false positives.
- It does not guarantee detection of every type of cyberattack.
- Encrypted traffic limits the amount of packet content that can be analyzed.
- The project is intended as an educational IDS prototype rather than a replacement for enterprise security systems.
- Testing should be performed only on networks and systems for which the user has authorization.

Chapter 4

Hardware and Software Requirements

4.1 High-Level System Architecture

The system follows a modular data-flow architecture:



4.2 Module Details

Module	Technology	Function
Packet Capture	Scapy	Captures network packets
Packet Parser	Python + Scapy	Extracts packet information
Traffic Analyzer	Python	Analyzes network behavior
Detection Engine	Python	Detects suspicious patterns
Risk Classifier	Python	Assigns risk levels

4.3 Hardware Requirements

S. No.	Name of Hardware	Specification
1.	Computer	Laptop or Desktop
2.	Processor	Intel i5 / AMD equivalent or higher
3.	RAM	Minimum 8 GB
4.	Storage	Minimum 10 GB available

4.4 Software Requirements

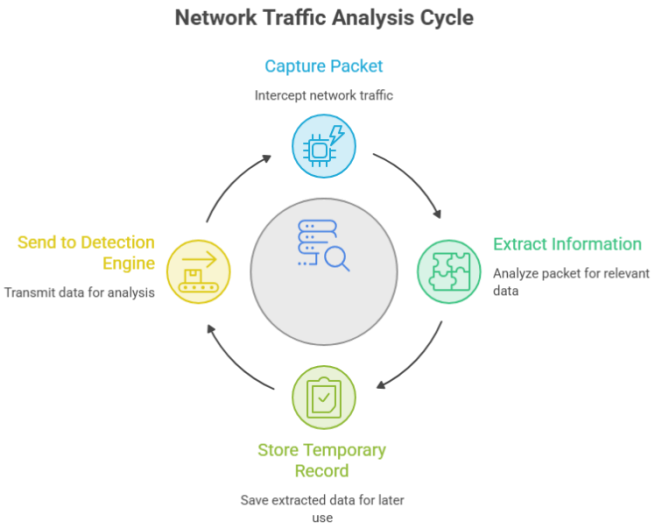
S. No.	Name of Software	Specification
1.	Operating System	Windows 10/11 or Linux
2.	Programming Language	Python 3.9 or above
3.	Packet Analysis	Scapy
4.	Database	SQLite
5.	GUI	Tkinter
6.	Web Dashboard	Flask, HTML, CSS, JavaScript
7.	Visualization	Matplotlib / Chart.js

Chapter 5

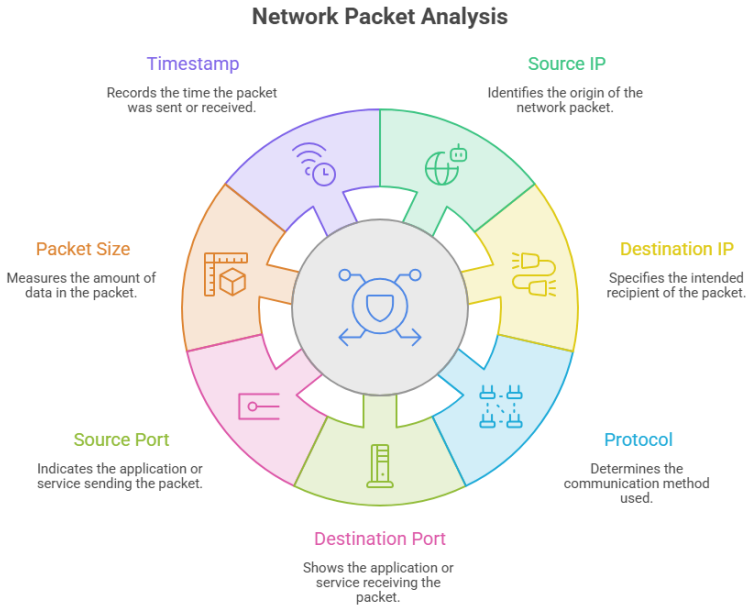
Possible Approach/ Algorithms

5.1 Packet Capture Pipeline

The system first captures packets from the selected network interface.



The information extracted may include:



5.2 Port Scan Detection Algorithm

The system maintains a record of destination ports contacted by each source IP.

Algorithm

1. Capture packet

2. Extract source IP

3. Extract destination IP

4. Extract destination port

5. Add port to source IP's port list

6. Count unique ports within time window

7. Compare count with threshold

8. If threshold is exceeded:

    Generate Port Scan Alert

9. Store alert in database

Example:

Source IP: 192.168.1.15

Ports:

22

23

25

53

80

110

135

139

443

445

...

Unique Ports > Threshold

↓

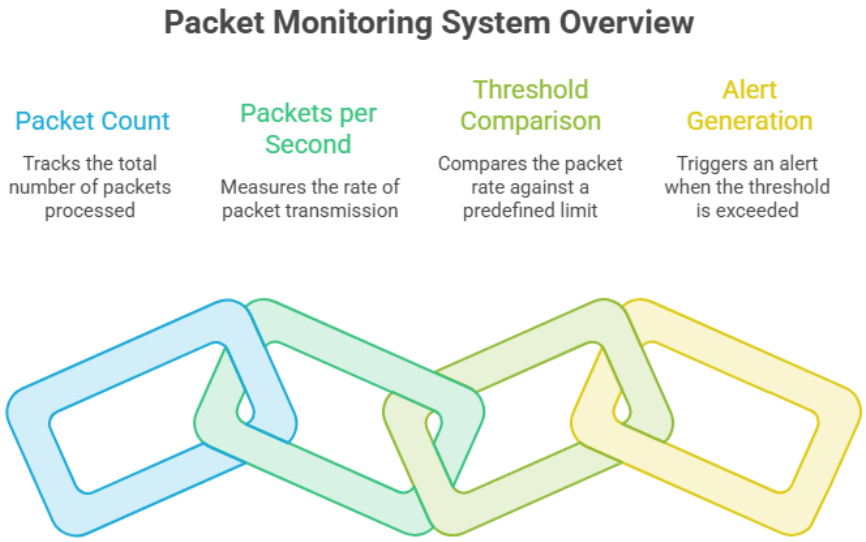
Possible Port Scan

↓

HIGH Risk Alert

5.3 Abnormal Traffic Detection

The system counts packets received from each source IP during a specific time interval.



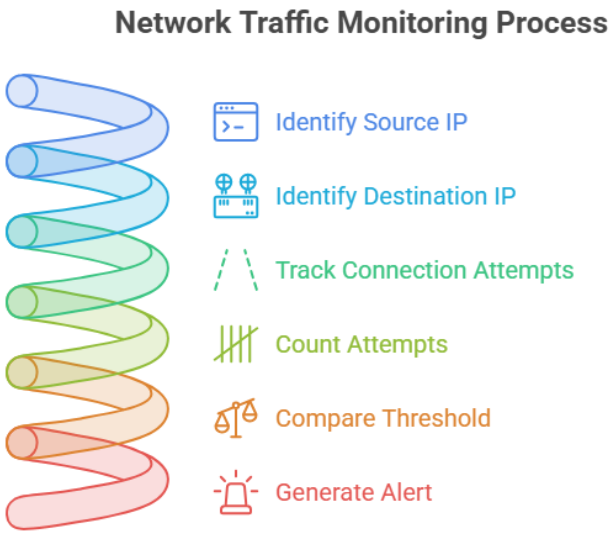
Example:
Source: 192.168.1.15
Packets in 10 seconds: 850

Threshold: 500

Result:
Possible Abnormal Traffic
Risk: HIGH

5.4 Repeated Connection Detection

The system tracks repeated connection attempts.

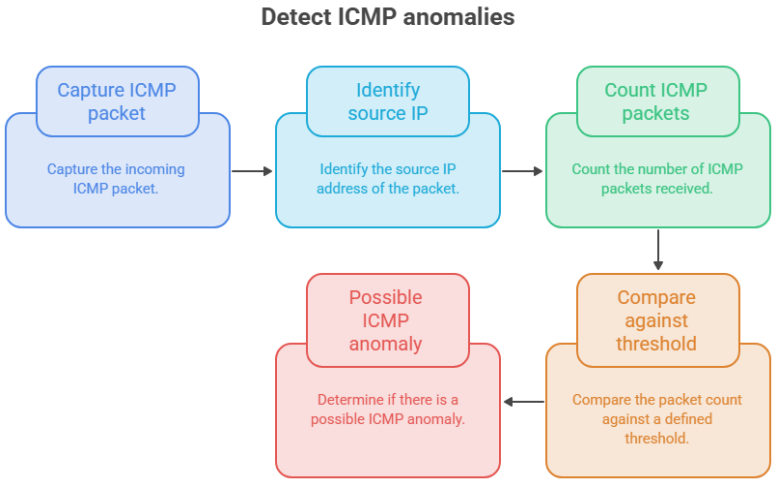


Example:
Source:
192.168.1.15
Target:
192.168.1.20

Attempts:
35
Status:
Suspicious

5.5 ICMP Anomaly Detection

The system monitors ICMP packets and counts their frequency.



This allows the prototype to demonstrate detection of unusually high ICMP traffic patterns without attempting to disrupt any system.

5.6 Risk Classification Algorithm

Each detected event can receive a risk level.



Example:
0–30 → LOW
31–70 → MEDIUM
71–100 → HIGH

The score can consider factors such as:

- Number of packets
- Number of unique ports
- Frequency of connections
- Number of detection rules triggered

5.7 Alert Generation

When a detection rule is triggered, the system creates an alert.
Example:
[ALERT]
Possible Port Scan Detected
Source : 192.168.1.15
Target : 192.168.1.20
Ports : 20-1000
Packets : 981
Risk : HIGH
Timestamp : 14:32:15
The alert is then stored in SQLite and displayed on the dashboard.

5.8 Database Storage

The system can maintain two primary tables.
Network Traffic
traffic_id
timestamp
source_ip
destination_ip
protocol
source_port
destination_port
packet_size

Alerts:
alert_id
timestamp
source_ip
destination_ip
attack_type
description
risk_level
status
This allows historical security events to be analyzed later.

5.9 Dashboard

The final dashboard can display:

==========

INTRUSION DETECTION SYSTEM

==========

System Status: MONITORING

Packets Captured 12,458
Suspicious Packets 531
Total Alerts 28
High Risk Alerts 5

RECENT ALERTS

[HIGH] Possible Port Scan
192.168.1.15 → 192.168.1.20

[MEDIUM] Abnormal Traffic
192.168.1.18

[LOW] Repeated Connections
192.168.1.25

References

1. Scapy Project. **Scapy Documentation**.
2. Python Software Foundation. **Python Documentation**.
3. SQLite. **SQLite Documentation**.
4. NIST. **Guide to Intrusion Detection and Prevention Systems (IDPS)**. NIST Computer Security Resource Center
5. Denning, D. E. (1987). **An Intrusion-Detection Model.** IEEE Transactions on Software Engineering, 13(2), 222–232.
6. Roesch, M. (1999). **Snort: Lightweight Intrusion Detection for Networks.** Proceedings of the 13th USENIX Conference on System Administration.
7. Open Web Application Security Project. **OWASP Documentation and Security Resources.**
   OWASP